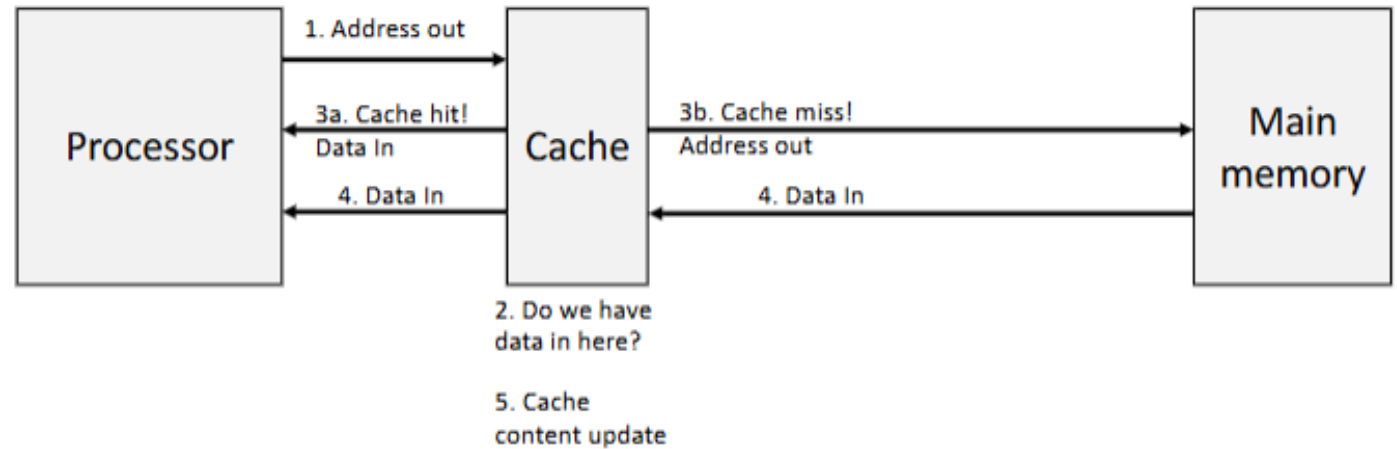


COMPE 271 Project C

Bryan Heraz

Background

- Modern CPUs have a memory Hierarchy
 - L1 = 384KB
 - L2 = 6.0MB
 - L3 = 32.0MB
 - Main Memory
- Cache Hit: data found in cache -> fast
- Cache Miss : data fetched from Main Memory -> slow
- System: AMD Ryzen 7600X 6 Core Processor



Project Goal

- Demonstrate cache memory efficiency effects when analyzing runtime performance
- Faster execution -> cache hits
- Slow Execution -> cache misses
- Two experiments:
 - Experiment 1: Array Size
 - Experiment 2: Change stride value, decides memory access pattern. Noncontiguous behavior

Algorithm Used

- Algorithm: Two Vector addition
 - $C[I] = A[I] + B[I]$
- All arrays allocated on heap using malloc.
- Every Element in array is read, computed, and stored in C
- Algorithm selected for its simplicity, highlights cache behavior

Functions Implementation

- `array()`
 - Created array and fills array A and B
 - Function called before time is recorded. Ensures fill time of array no included in measurements
- `get_time()`
 - Measures wall clock time
 - Utilizes `Clock_MONOTONIC`
- `contiguous_acces()`
 - Sequential vector addition
- `Strided_acces()`
 - Non-contiguous vector additions
 - Stride value determine next element accessed

```
// get_time() function that returns time into nanoseconds for cache performance testing
static uint64_t get_time(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts); // time conversion to nanoseconds
    return (uint64_t)ts.tv_sec * 1000000000ull + (uint64_t)ts.tv_nsec;
}
```

Get_time() Function

- Clock_gettime() to use CLOCK_MONOTONIC, found in POSIX 2008 standard function
- Value stored in timespec address
- Tv_sec and tv_nsec accessed with member access operator
- Time converted to nanoseconds.
- Function is called twice, before and after. Records start and end time.

Contiguous_access() Function

```
void contiguous_access(int *A, int *B, int *C, int size)
{
    for (int i = 0; i < size; i++)
    {
        C[i] = A[i] + B[i];
    }
}
```

- Vector addition performed continuously
- Each element access one by one
- Good spatial location, helped test large arrays (main focus for experiment 1)

Strided_access() Function

- Non-contiguous vector addition
- Fixed size, Large array used to show performance difference
- Jumps through array bases on stride value
- 100000000 operations for every test, make test fair since larger stride reduce operations
- $\text{index} = (\text{index} + \text{stride}) \% \text{size}$ formula. Make sure index stay within array size since large operations are implements

```
// Strided_access() function, iterates through the array with name  
void strided_access(int *A, int *B, int *C, int size, int stride)  
{  
    int index = 0;  
    for (int i = 0; i < OP; i++)  
    {  
        C[index] = A[index] + B[index];  
        index = (index + stride) % size; // control pattern  
    }  
}
```

```
int sizes[] = {1000,
               10000,
               100000,
               1000000,
               10000000,
               100000000};
```

```
for (int i = 0; i < number_size; i++)
{
    int current_size = sizes[i]; // creating array of current size

    int *A = malloc(current_size * sizeof(int)); // allocating memory for array
    int *B = malloc(current_size * sizeof(int)); // allocating memory for array
    int *C = malloc(current_size * sizeof(int)); // allocating memory for array

    array(A, current_size); // creating array
    array(B, current_size); // creating array

    uint64_t time_started = get_time(); // start time recorded
    contiguous_access(A, B, C, current_size); // contiguous function called to test perform
    uint64_t time_ended = get_time(); // end time recorded

    printf("%-10d %20.3f\n", current_size, (double)(time_ended - time_started) / 1e6); // p
    free(A); // f
    free(B); // f
    free(C); // f
}

printf("\n\n");
```

First Experiment

-
- For loop uses to test each size
 - Time recorded before and after algorithm is performs
 - Wall time Calculated
 - Memory deallocation

Second Experiment

```
int strides[] = {1, 2, 4, 8, 16, 32, 64, 128};
```

```
int fixed_size = 32000000;           // fixed size of array, above size of L3 cache
int *A = malloc(fixed_size * sizeof(int)); // allocating memory for array
int *B = malloc(fixed_size * sizeof(int)); // allocating memory for array
int *C = malloc(fixed_size * sizeof(int)); // allocating memory for array

array(A, fixed_size); // create array
array(B, fixed_size); // create array

for (int i = 0; i < number_stride; i++)
{
    int current_stride = strides[i]; // current stride using in stride array

    uint64_t time_started = get_time();           // start time recorded
    strided_access(A, B, C, fixed_size, current_stride); //
    uint64_t time_ended = get_time();             // end time recorded

    printf("%-10d %20.3f\n", current_stride, (double)(time_ended - time_started) / 1e6); //
}

free(A); // frees memory
free(B); // frees memory
free(C); // frees memory
return 0;
```

List of stride use to test

All test will have same Fixed size array

Four loop use to test each stride, wall time record for every test

Memory deallocated when is done

RESULTS

Contiguous Access Performance (small to large array)

Array Size Time (ms)

1000 0.001
10000 0.020
100000 0.177
1000000 1.886
10000000 21.826
100000000 208.185

Strided Access performance (fixed size, different strides)

Stride Time (ms)

1 511.851
2 468.067
4 469.619
8 475.701
16 654.041
32 1156.090
64 1100.356
128 1524.316

Conclusion

- Size of array Test (Contiguous Access)
 - Results are promising, time increase as size increases
 - Arrays fit in caches perform under 0.01ms, Cache hits
 - Time increase when data exceed cache, required CPU to fetch data in main memory
- Stride Access
 - As Stride increase, execution time increase.
 - Cache misses increase
 - Large stride skip cache lines, forces CPU to fetch data in main memory